Smart Text Classifier - Internship Report

1. Project Title

Smart Text Classifier: classifying short text into Complaint, Inquiry, Feedback or Other.

2. Introduction / Objective

The goal was to build a small AI application that assigns short messages to one of four categories. I wanted a complete, understandable project: data, model, evaluation, a user interface and tests, using only fundamentals I can explain.

3. Approach

Dataset -> preprocessing -> TF-IDF -> Logistic Regression -> evaluation -> saved model -> Streamlit app. TF-IDF and the classifier are combined in one scikit-learn Pipeline, so the saved model takes raw text directly.

4. Dataset

I wrote the dataset myself for this project; it is not from an external source. It has 240 sentences in data/dataset.csv (text, category), 60 per class. Validation found no missing values and no duplicates. Sentences cover many everyday topics so that the model cannot rely on one repeated pattern.

5. Preprocessing

Lowercasing, removing apostrophes and punctuation, and normalising whitespace. Non-text and empty values become an empty string and are rejected safely. I kept stop words because words like "how" and "not" help separate Inquiry and Complaint. The cleaning function is used inside the TF-IDF step, so training and prediction clean text identically.

6. TF-IDF

Computers need numbers, so TF-IDF represents each sentence as a vector of word weights. TF (term frequency) is how often a word appears in the sentence; IDF (inverse document frequency) lowers the weight of words common in many sentences. I used single words and word pairs and sublinear_tf=True. The vectorizer is fitted on training data only to avoid data leakage.

7. Machine Learning Model

Logistic Regression (C=10, max_iter=1000). It learns a weight for each TF-IDF feature per class and outputs probabilities, which the app shows as confidence. It is fast, works well on sparse text features and is easy to explain, which suits a small dataset better than a neural network.

8. Evaluation

80/20 stratified split (192 train / 48 test, random_state=42). Real test results: accuracy 0.7292, weighted F1 0.7293, macro F1 0.7293, weighted precision 0.7429, weighted recall 0.7292. Training accuracy was 1.00 and 5-fold cross-validation accuracy on the training set was 0.62 +/- 0.07, which indicates overfitting. Inquiry was the easiest class (F1 0.88); Complaint was the hardest (F1 0.61), often confused with Feedback. On 12 new sentences, 11 were classified as expected; the miss was "The mountains look lovely under the snow" (predicted Feedback).

9. Key Technical Decisions

* Pipeline to prevent data leakage and make saving/loading simple.
* Logistic Regression instead of deep learning: small data, need for explainability.
* Weighted F1 as the main F1 (accounts for class sizes); macro F1 also reported. With balanced classes they are equal.
* Cleaning inside the vectorizer so prediction cannot accidentally use different preprocessing.
* Cross-validation added because 48 test sentences give a noisy estimate.
* Not tuning on the test set: I compared a few settings with cross-validation on training data only; they made no real difference, so I kept the simple configuration.

10. Challenges

Writing a diverse dataset by hand was harder than expected, especially separating Complaint from Feedback and keeping "Other" from looking like the rest. Results were lower than I first hoped; the evaluation showed that the dataset size, not the model settings, is the main limit. Environment note: the Streamlit app was written and checked with a stubbed run, but the final visual check should be done locally.

11. Limitations

Small, single-author dataset; overfitting; bag-of-words ignores meaning; sentences can belong to several classes; always chooses one of four classes (no "unknown"); English only; confidence values are estimates.

12. Future Improvements

Collect a larger real-world dataset; compare other models (Linear SVM, Naive Bayes) with proper validation; add a low-confidence warning; support typos and other languages.

13. Conclusion

The project is a working, honest baseline for short-text classification. It reached about 73% test accuracy on a small dataset, and it taught me how preprocessing, TF-IDF, a linear model and evaluation fit together, including why training accuracy alone is misleading.